

Depth-Swap Upward-Motion Artifact: Discovery, Fix, and Lessons

Gene Stoner — 2026-04-11

1. The artifact

All sessions of Exp_DecoupledDots_005m (n=2048), Exp_DepthColorLinked_005m (n=1024), and Exp_BothFar_005m (n=512) collected prior to 2026-04-11 contain a systematic stimulus artifact in any condition involving a depth swap at tStart (Z, CZ, ZdA, ZdB).

Direction convention note. The stimulus code uses math convention: 0° = rightward (+X in local space), 90° = upward (+Y). All direction values in this document follow that convention.

The symptom. In Z and CZ conditions, approximately 50% of all wrong responses are at 90° (upward), regardless of the true translation direction. In the no-swap (N) and color-only (C) conditions, upward accounts for only ~4–6% of wrong responses — the baseline expected from random errors. For the downward heading (270°), 61% of all responses in Z condition say upward — the wrong direction. For the upward heading (90°), Z condition accuracy is *elevated* relative to N (45% vs 36%) because the artifact and the signal coincide.

Condition	UP (90°) share of wrong responses	n wrong
N	4.3%	324
C	6.0%	316
Z	49.6%	415
CZ	50.0%	398

The artifact is present across all sessions, both rotation configurations (RotCfg 0: 52%, RotCfg 1: 47%), CUED and UNCUED arms (51% vs 49%), and both depth planes (DFD=Near: 48%, DFD=Far: 52%). It is not moderated by any experimental factor. It is a stimulus bug, not a perceptual phenomenon.

Scaling. The artifact magnitude scales with the total depth-change impulse:

- DecoupledDots Z (100% swap, 0.10m total change): 50% UP in wrong responses
- BothFar Z (100% swap, 0.05m total change — both planes behind fixation): 44% UP
- DepthColorLinked ZdA/ZdB (50% of dots swap, 0.10m per dot): 26% / 22% UP

2. Root cause

`StimulusBuilder.ApplyDepthOffsets()` shifts every dot by `transform.forward * z` at each frame, where `z = ±depthOffset_m` depending on the dot's assigned depth plane. In conditions with no depth swap, this is applied consistently every frame and produces no net displacement. In the Z/CZ conditions, all dots simultaneously change depth plane at tStart (frame 78), causing a one-frame shift of $\pm 0.10\text{m}$ along `transform.forward`.

The bug: `transform.forward` **is the forward axis of the StimulusBuilder GameObject in world space, which is not necessarily aligned with the camera's optical axis.** If the StimulusBuilder's forward axis is pitched slightly upward relative to the camera's line of sight — as is the case here — the 0.10m depth displacement along that axis has a vertical component in screen space. The resulting apparent shift occurs in a single frame (~13ms at 75 Hz):

```
apparent_upward_shift = 0.10m x sin(θ) [in screen-space meters at 2m distance]
apparent_angular_shift = arctan(0.10 x sin(θ) / 2.0) ≈ 2.86deg x sin(θ)
apparent_angular_velocity = 2.86deg x sin(θ) / 0.013s = 220deg x sin(θ) per second
```

For even a 1° pitch misalignment: apparent velocity ≈ 3.8°/sec, already larger than the translation signal (2.26°/sec). For 5°: ~19°/sec. A modest misalignment between the StimulusBuilder's world orientation and the observer's gaze direction is sufficient to produce an impulse that completely dominates the direction percept.

3. The fix

One line changed in `StimulusBuilder.ApplyDepthOffsets()`:

Before (buggy):

```
Vector3 zVec = transform.forward * z;
```

After (fixed):

```
Vector3 depthAxis = (Camera.main != null)
    ? Camera.main.transform.forward
    : transform.forward;
Vector3 zVec = depthAxis * z;
```

Using `Camera.main.transform.forward` ensures the depth offset is applied along the observer's actual optical axis regardless of the StimulusBuilder's world-space orientation. The fallback to `transform.forward` fires only if `Camera.main` is null (e.g., in editor preview), preserving prior behaviour for in-editor inspection.

Residual effect after fix. A small perspective-induced radial expansion will remain: when dots jump from one depth plane to another, their projected screen angles change slightly because the viewing distance changes. For a dot at 2° eccentricity and a 0.10m depth change at 2m viewing distance, this radial shift is ≈ 0.05° — about 2% of the translation signal and radially symmetric (no directional bias). This is physically correct behaviour, not a bug.

Affected assets (all renamed with `\_v2` suffix in `experimentName` field):

- `Exp_DecoupledDots_005m` → `DecoupledDots_005m_v2`
- `Exp_DecoupledDots_Inv_005m` → `DecoupledDots_Inv_005m_v2`
- `Exp_DepthColorLinked` → `DepthColorLinked_005m_v2`
- `Exp_BothFar_005m` → `BothFar_005m_v2`
- `Exp_DepthSwapCtrl` → `DepthSwapCtrl_005m_v2`

Any TSV with `experimentName` containing `_v2` was collected with the fixed code. Any TSV without `_v2` is pre-fix and should be excluded from quantitative analysis of depth-swap conditions.

Verification before collecting new data. Run one session of `DecoupledDots_005m_v2`, then run `depth_swap_artifact_analysis.py`. Confirm that the UP (90°) share of wrong responses in Z condition falls to ≤ 10% (the N condition baseline). Only then proceed with systematic data collection.

4. Impact on existing findings

The cueing advantage (CUED – UNCUED) is preserved throughout the contaminated data, because the artifact affects both arms equally (51% vs 49% UP in wrong responses). The four main qualitative conclusions from the pre-fix data are robust:

1. **Depth-plane continuity matters for cueing.** Z/CZ conditions reduce the cueing advantage vs N. This is real. The artifact depresses absolute accuracy in both CUED and UNCUEd arms, but the difference survives.
2. **Color is null.** The C condition has no artifact (4–6% UP in wrong responses, same as N). The color-null finding is fully clean.
3. **ZdNoi > ZdCoh.** Both DCL conditions have similar UP bias (26% vs 22%), yet their cueing outcomes differ by ~19pp. The artifact cannot explain the dissociation.
4. **Far > Near asymmetry.** The artifact is equal for both depth planes. The asymmetry is real.

What is wrong: absolute accuracy levels in Z/CZ are incorrect (depressed for 7/8 headings, inflated for the UP heading). The quantitative magnitude of depth disruption is inflated. After rerunning with fixed code, the Z and CZ effects will likely be somewhat smaller.

5. Why the traces missed this

The existing trace figures (decoupled_*_traces.pdf, depthcolorlinked_combined_condensed.pdf) do not show this artifact for a fundamental reason: **they represent dot positions in 2D local-space coordinates, not in screen space as the observer's eyes see them.**

The traces are generated by simulating the dot physics (rotation and translation) in the StimulusBuilder's local XY plane. Depth is represented symbolically — filled circles for one plane, open circles for the other. The depth offset (ApplyDepthOffsets) is never included in the simulation. The result is a figure that correctly represents the intended design (which field is near, which is far, when they swap) but is blind to any artifact in how the depth offset is physically rendered.

This is the core gap: **there is no step in our verification pipeline that checks what the observer's eyes actually see.**

6. Stereo-projected traces: the solution

To make traces that reflect exactly what subjects see, we need to compute the perspective projection for each eye at each frame.

The math. For a dot at local-plane position (x, y) [meters] and depth offset z [meters positive = away from viewer], at viewing distance D = 2.0m:

```
z_world = D + z_depth          # actual viewing distance to dot

# Left eye (camera offset -IPD/2 along screen X):
x_left  = (x + IPD/2) / z_world  # projected x [radians ~= degrees/57.3]
y_left  = y / z_world

# Right eye:
x_right = (x - IPD/2) / z_world
y_right = y / z_world          # y projection is identical for both eyes

# Binocular disparity (positive = uncrossed = "far"):
disparity = x_left - x_right = IPD / z_world
```

What to plot. A stereo-correct trace shows:

- Left-eye projected position as a solid line
- Right-eye projected position as a dashed line
- At a depth-swap frame, both lines jump — and the jump direction and magnitude is the artifact

What the pre-fix artifact looks like in stereo traces. When `transform.forward` is pitched upward by θ , the depth-swap creates an upward shift of $0.10 \times \sin(\theta)$ meters along the screen vertical, in addition to the expected small radial expansion. This appears as a vertical jump in both the left- and right-eye traces at `tStart`, identical in both eyes (not a disparity change — a common-mode screen shift). The post-fix traces show only the tiny radial expansion (both eyes shift slightly outward at the aperture edges, negligible at the center) — correct physics with no directional bias.

Implementation. The script `stereo_trace_artifact_demo.py` (in `Tools/Analysis/`) implements this. It takes a single Z-condition trial, simulates dot positions frame-by-frame, and renders three panels:

1. 2D local-space traces (what the old traces showed — artifact invisible)
 2. Cyclopean screen-space traces with pre-fix `transform.forward` pitch = 5° (artifact visible: upward jump at `tStart`)
 3. Cyclopean screen-space traces with post-fix camera-aligned axis (clean: tiny radial expansion only)
-

7. General lesson: verification pipeline

How this bug was found. Observer GS reported "jerky upward motion" on a subset of trials. This prompted analysis of the wrong-response direction distribution, which revealed the UP spike. Without the subjective report, the bug would have remained undetected. The data revealed it only because the effect was strong enough to dominate the response in ~40% of trials.

Weaker artifacts might not be caught this way. A small misalignment producing a $0.5^\circ/\text{sec}$ artifact would not show up in direction reports but could shift thresholds. The lesson: behavioral data analysis is a lagging indicator. We need a leading indicator.

Recommended additions to the verification pipeline:

1. **Stereo-projected traces** for any new stimulus design — run before collecting data. Plot projected positions for each eye; any depth-change frame should show only radial expansion, not directional drift.
2. **Single-frame inspection tool** — render the dot positions at `tStart`, `tStart+1`, `tStart-1` in screen space for each eye. A step-change frame should be inspectable directly.
3. **Response direction monitor** — after every session, auto-run the wrong-response direction distribution. Flag any direction with >15% of wrong responses as anomalous (vs the ~12.5% uniform baseline).

The last one costs nothing to add and would have caught this bug after the very first session.